S2S-04: Step 4 — Prepare: Export and Pre-Stage

> **Series:** S2S — SaaS to SaaS Migration | **Notebook:** 4 of 9 | **Phase:** Upgrade | **Step:** Prepare | **Created:** March 2026 | **Last Updated:** 10/15/2026

Overview

With the target tenant architecture designed (Step 3), this step builds the foundation. Preparation is the longest step in any SaaS-to-SaaS migration — it involves exporting configuration from the source tenant, provisioning the target tenant, deploying IAM, installing ActiveGates, preparing Kubernetes operators, and freezing the source tenant to prevent drift during execution.

Every task in this notebook must be completed and validated before Step 5 (Execute) begins. Skipping preparation tasks is the primary cause of migration rollbacks.

S2S Migration Framework

Phase	Steps	Focus
Plan	1. Discover → 2. Strategize → 3. Design	Understand what exists, choose approach, design target
Upgrade	**4. Prepare** → 5. Execute → 6. Integrate	Build target, move config, connect integrations
Run	7. Expand → 8. Enable → 9. Optimize	Roll out agents, enable teams, tune and decommission

> **You are here: Step 4 — Prepare.** You have designed the target architecture (Step 3). Now you export configuration, provision the target tenant, and stage all prerequisites before execution begins.

Table of Contents

1. Target Tenant Provisioning
2. SSO and IAM Setup
3. Monaco Bulk Export
4. ActiveGate Provisioning
5. Kubernetes Operator Preparation
6. Configuration Freeze
7. Rollback Procedure
8. Step Completion Checklist

Prerequisites

Requirement	Details
Step 3 Complete	Target tenant architecture designed — IAM, buckets, OpenPipeline, deployment order documented
Source Tenant Access	Admin access with `ReadConfig`, `settings.read`, `activeGates.read` scopes
Target Tenant Provisioned	Dynatrace SaaS environment with Grail enabled
Monaco CLI	`monaco v2.x` installed and configured
Terraform	`terraform >= 1.0` with Dynatrace provider `~> 1.91`
Helm	`helm >= 3.x` for Kubernetes operator deployment
Change Window	Approved change window for configuration freeze

Order of Operations

This notebook covers **operations 1–3** of the Upgrade phase:

#	Operation	Section	Dependency
1	**Provision target tenant and configure SSO/IAM**	Sections 1–2	Step 3 design deliverables
2	**Export configuration from source tenant**	Section 3	Source tenant access
3	**Deploy ActiveGates and prepare K8s operators**	Sections 4–5	Target tenant provisioned
4	Import configuration to target tenant	Step 5	Operations 1–3 complete
5	Redirect agents to target tenant	Step 5	Configuration imported
6	Validate data flow in target tenant	Step 5	Agents redirected
7	Reconnect cloud integrations	Step 6	Agents reporting
8	Migrate dashboards, workflows, and extensions	Step 6	Integrations connected

> **Bold** operations are covered in this notebook. Operations 4–8 are covered in Steps 5 and 6.

1. Target Tenant Provisioning

The target tenant must be provisioned and configured with baseline settings before any configuration is imported.

Provisioning Checklist

Item	Action	Status
Hosting cloud and region	Choose the hosting cloud and region deliberately (same region as source if relocation) — see *Choosing the Target's Hosting Cloud* below	☐
Environment activation	Activate SaaS environment via Dynatrace Account Management	☐

Item	Action	Status
Grail enablement	Verify Grail is enabled (default for new SaaS tenants)	☐
API token creation	Create token with `WriteConfig`, `ReadConfig`, `settings.write`, `settings.read` scopes	☐
OAuth client creation	Create OAuth client for account-level API access (IAM management)	☐
Platform version	Record both environments' Dynatrace versions — SaaS releases roll out to environments in stages, so source and target can briefly differ; check version-gated features on the target before relying on them	☐

Region Considerations

Scenario	Region Strategy
Relocation	Same region as source (minimize latency change)
Consolidation	Region closest to majority of monitored infrastructure
Hosting-cloud change	A region on the new cloud provider — for Azure, one of the regions Dynatrace offers on request (below)
Compliance-driven	Region that satisfies data residency requirements (EU, US, APAC)

Choosing the Target's Hosting Cloud

A SaaS environment runs on one cloud provider. Dynatrace documents that *"Data is stored in Amazon Web Services (AWS), Microsoft Azure, or Google Cloud data centers."* Moving the environment from an AWS-hosted cluster to an Azure-hosted one is therefore a **new target environment**, provisioned on Azure, followed by this whole series — not a setting on the existing environment.

Azure-hosted regions are listed as East US, West US 3, West Europe, Canada Central, UAE, Switzerland North and Australia East, each footnoted *"Available on request. Talk to your Dynatrace sales contact."* Start that conversation before anything else in Step 4 is scheduled.

There are two ways to get an Azure-hosted target:

	Standard Dynatrace SaaS on an Azure region	Azure Native Dynatrace Service
How it is provisioned	Through your Dynatrace account team, in an Azure region *"Available on request"*	From the Azure portal, via the Azure Marketplace — *"available via a private offer"*
Billing	Your existing Dynatrace agreement	*"your Dynatrace license consumption becomes a part of your regular Azure bill"*
Account and environment	A new environment, set up with your account team (settle with them how it relates to your existing account)	*"The integration will create a new Dynatrace environment and account; it can't run on an existing Dynatrace SaaS environment."* The Azure user who creates the first resource *"becomes the owner of the Dynatrace account"*
Region	Chosen with the account team	*"The Dynatrace environment is created in the same Azure region in which you create the Dynatrace resource."*
Identity	SAML SSO — Microsoft Entra ID is a documented IdP	*"Enable SSO through Azure Active Directory"*; *"The integration works across a single Entra ID environment."*
Cloud monitoring on the target	Azure Cloud Platform Monitoring (Clouds app) as documented	**Open question:** the Azure Native page is labelled Dynatrace Classic and says nothing about Azure Cloud Platform Monitoring or the Clouds app on an environment it creates. Confirm with Dynatrace before choosing this path

A consideration the docs do not settle, raised here as a softened recommendation: in community practice, the choice between the two paths is mostly **commercial** — a new account (Azure Native) means a separate account structure, a separate IAM and account-management surface, and consumption billed through Azure rather than your existing DPS agreement. Bring your Dynatrace account team and your Azure commercial owner into the decision together, and settle how the source environment's commitment and the target's consumption overlap during the migration window.

> **Sources:**
>
> - Data security controls (DT docs) — *"Data is stored in Amazon Web Services (AWS), Microsoft*

> *Azure, or Google Cloud data centers."*; Azure regions *"Available on request. Talk to your Dynatrace sales contact."*; *"Environments hosted on Azure use dedicated Azure storage accounts."*
>
> - Azure Native Dynatrace Service (DT docs) — *"The integration will create a new Dynatrace environment and account; it can't run on an existing Dynatrace SaaS environment."*; *"The integration works across a single Entra ID environment."*
> - Azure Cloud Platform Monitoring (DT docs)

Pre-Migration Baseline: Entity Counts

Before any migration begins, capture entity counts from the source tenant. These serve as validation targets after agent cutover in Step 5.

> **Compare like with like.** The three host surfaces do not agree. Over the same 24 hours on a validation tenant (10/01/2026), `smartscapeNodes "HOST"` returned **7** hosts, `fetch dt.entity.host` returned **33**, and the billing usage events named **39**. FAQ-16 explains why Smartscape and the classic entity store differ. Pick one surface, use it in both the source and the target, and run the same query on both. For license sizing, read host consumption from the billing usage events (ADOPT-02 § 6) rather than from either entity count.

```
// Source tenant: baseline host count (Smartscape)
smartscapeNodes "HOST", from:-7d
| summarize host_count = count()
| fieldsAdd entity_type = "Hosts"

// Run the SAME query in the target after cutover — do not compare a
Smartscape count with a
// classic one. Classic fallback: fetch dt.entity.host | summarize host_count
= count()
// (the classic store can retain hosts Smartscape no longer lists).
```

```
// Source tenant: baseline service count (Smartscape)
smartscapeNodes "SERVICE", from:-7d
| summarize service_count = count()
| fieldsAdd entity_type = "Services"

// Classic fallback: fetch dt.entity.service | summarize service_count =
count()
```

```
fetch dt.entity.process_group, from:-7d
| summarize pg_count = count()
| fieldsAdd entity_type = "Process Groups"
```

```
// Source tenant: Grail bucket inventory for migration planning
fetch dt.system.buckets
| fields display_name, dt.system.table, retention_days,
estimated_uncompressed_bytes
| fieldsAdd size_gb = estimated_uncompressed_bytes / 1073741824.0
| sort size_gb desc
```

> **Record these counts.** After agent cutover in Step 5, you will re-run these queries against the target tenant and compare. As a rule of thumb from community practice, counts within about 5% of the baseline indicate a clean wave; counts below about 90% point to missing agents or misconfigured host groups.

2. SSO and IAM Setup

IAM is the first configuration deployed to the target tenant. Users need access to validate subsequent deployments, and IAM policies reference resources that will be created in later steps — but the group structure and base policies can be deployed now.

SAML/SSO Configuration

Step	Action	Notes
1	Navigate to **Account Management** → **Identity & Access Management** → **SSO**	Target tenant
2	Configure SAML identity provider (same IdP as source if consolidation) — Microsoft Entra ID has a documented Dynatrace SAML configuration; an Azure Native environment uses Entra ID SSO from the start	Copy metadata URL from IdP; create a **new** enterprise application for the target. Dynatrace requires the **entire SAML message** to be signed — Entra's default for most gallery apps signs only the assertion, so set *Sign SAML response and assertion*

Step	Action	Notes
3	Map IdP groups to Dynatrace groups	Use the IAM design from Step 3
4	Enable SSO enforcement (after initial admin access is confirmed)	Do not lock out admin accounts
5	Test SSO login with at least two different group memberships	Verify policy inheritance

Rebuild the IP Allowlist

If the source environment restricts access with an IP allowlist, rebuild it on the target before users and automation cut over. It is configured per environment, and it blocks inbound access to the UI and API: *"If a user's IP isn't contained in the IP allowlist, they're blocked from accessing and using the latest Dynatrace web UI and API."* Include the addresses of your CI/CD runners and any automation that calls the target API.

The reverse direction — **egress** addresses of the new Azure-hosted cluster that your firewalls or webhook receivers may need to admit — is **unverified**: no primary source for them was found for this update. Ask Dynatrace for them rather than reusing the AWS-hosted source's addresses.

> **Sources:** SAML (DT docs) — *"The entire SAML message must be signed (signing only SAML assertions is insufficient and generates a 400 Bad Request response)."*; Advanced certificate signing options in a SAML token (Microsoft Learn) — *"default option set for most of the gallery applications"*; IP allowlist (DT docs), Azure SAML configuration for Dynatrace (DT docs) — *"Follow the examples below to configure Azure as the SAML identity provider (IdP) for Dynatrace SSO."*
>
> **Warning:** Do not enable SSO enforcement until you have confirmed that at least one admin account can log in via SSO. A misconfigured SSO with enforcement enabled will lock all users out of the tenant.

Terraform IAM Deployment

This series deploys IAM with Terraform (Monaco's `monaco account deploy` is the alternative, with an OAuth client). Deploy the group structure and base policies designed in Step 3:

```
# Initialize Terraform with Dynatrace provider
cd terraform/iam
terraform init

# Plan IAM deployment
terraform plan -var-file="target-tenant.tfvars"

# Apply IAM configuration
terraform apply -var-file="target-tenant.tfvars"
```

IAM Deployment Order

Order	Resource	Terraform Resource Type
1	Groups	`dynatrace_iam_group`
2	Policies	`dynatrace_iam_policy`
3	Policy bindings	`dynatrace_iam_policy_bindings`

> **Note:** For the initial deployment, create policies with broad `ALLOW` statements. After all configuration is imported (Step 5) and validated, tighten policies with `WHERE` clauses that reference specific buckets, schemas, and security contexts.

3. Monaco Bulk Export

Monaco `download` exports all configuration from the source tenant into a structured directory. This export becomes the input for `monaco deploy` in Step 5. The SaaS Upgrade Assistant is documented for a Managed source only — *"SaaS Upgrade Assistant imports your Dynatrace Managed environment configuration"* (SaaS Upgrade Assistant (DT docs)) — so plan on `monaco deploy`. As a field practice, the same export can be packaged as a `.tar.gz` and imported through the assistant on the target; **S2S-10** §1 builds that package and lists what to rehearse first.

Export Commands

```
# Set environment variables
export DT_SOURCE_URL="https://&lt;source-env-id&gt;.live.dynatrace.com"
export DT_SOURCE_TOKEN="dt0c01.xxx..."

# Full export of source tenant
# --token takes the NAME of the environment variable, not the token itself
monaco download \
  --url "$DT_SOURCE_URL" \
  --token DT_SOURCE_TOKEN \
  --output-folder ./export \
  --force

# Export Settings 2.0 objects only
monaco download \
  --url "$DT_SOURCE_URL" \
  --token DT_SOURCE_TOKEN \
  --output-folder ./export-settings \
  --only-settings \
  --force
```

`--token` names an environment variable — the Monaco command reference: *"The name of the environment variable that contains the API token (classic Dynatrace only)."* Passing `"$DT_SOURCE_TOKEN"` hands Monaco the secret where it expects a variable name. Platform configurations (documents, automations, buckets, segments, SLOs, OpenPipeline) are downloaded with a platform token (`--platform-token`) or an OAuth client (`--oauth-client-id` and `--oauth-client-secret`), each likewise naming an environment variable. To narrow a download, use `--only-settings` , `--only-apis` , `--only-slo-v2` (and the other `--only-*` flags), `--settings-schema <schema>` or `--api <api>` . OpenPipeline configuration is Settings 2.0 objects: export it with `--settings-schema` and the `builtin:openpipeline.*` schemas, plus `--admin-access` to include pipelines other users own (S2S-07 §1). The command reference marks `--only-openpipeline` as **Deprecated**.

> **Sources:** Monaco CLI commands (DT docs), monaco download flags, v2.30.0 (Dynatrace GitHub).

Automated Export

For a scripted export — platform-aware Monaco binary download with checksum verification, a short-lived read-only export token that is revoked after the download, and a

timestamped export folder ready for `monaco deploy` — see **S2S-10: Migration Scripts**.

Export Directory Structure

After `monaco download`, the export directory contains:

```
export/
├── project/
│   ├── alerting-profile/          # Alerting profiles
│   ├── anomaly-detection-*/       # Anomaly detection rules
│   ├── auto-tag/                  # Auto-tag rules
│   ├── dashboard/                 # Classic dashboards
│   ├── management-zone/           # Management zones
│   ├── notification/              # Notification integrations
│   ├── request-attributes/        # Request attributes
│   ├── slo/                       # SLO definitions
│   ├── synthetic-*/               # Synthetic monitors
│   └── ... (50+ config types)
├── manifest.yaml              # Monaco manifest
└── delete.yaml                # Deletion tracking
```

Export Validation

After export, validate the output before proceeding:

```
# Count exported configuration items
find ./export/project -name "*.json" -o -name "*.yaml" | wc -l

# Verify the manifest renders (offline dry-run). Add the target environment to
# export/manifest.yaml first — see S2S-05 §2 — then:
monaco deploy ./export/manifest.yaml --environment target-tenant --dry-run

# Check for entity ID references that need remapping
grep -r "HOST-\|SERVICE-\|PROCESS_GROUP-" ./export/project/ | wc -l
```

> **Important:** Monaco download **cannot export** the following:
> - Cloud provider credentials (AWS IAM roles, Azure service principals, GCP service account keys)
> - Account-level IAM (groups, policies, bindings) — a plain `monaco download` skips it; use `monaco account download` (OAuth client) or Terraform
> - ActiveGate tokens and connection info
> - Synthetic private location credentials

> These must be recreated manually in the target tenant.

4. ActiveGate Provisioning

ActiveGates in the source tenant cannot be "moved" to the target tenant. New ActiveGates must be deployed and connected to the target tenant. The source ActiveGates remain operational until cutover is validated.

ActiveGate Sizing

A SaaS target uses **Environment ActiveGates** only — Cluster ActiveGates belong to Dynatrace Managed. Plan Environment ActiveGates by purpose, and size them with **FAQ-10** (ActiveGate sizing and scaling):

Purpose	Typical modules	Notes
Routing / API	OneAgent routing, API	Needed where hosts cannot reach the SaaS endpoint directly
Extensions	Extension Execution Controller	Remote extensions run on an ActiveGate group
Synthetic	Synthetic (private location)	One per private location; recreated in the target

> **Sources:** ActiveGate overview (DT docs) — *"If you are using the Dynatrace SaaS solution, you only need to install an Environment ActiveGate."*

Deployment Steps

Step	Action	Validation
1	Download ActiveGate installer from target tenant	Verify installer matches target tenant ID
2	Deploy to designated hosts (same hosts or new hosts)	ActiveGate appears under `smartscapeNodes "ACTIVEGATE"` in the target (query below)

Step	Action	Validation
3	Assign ActiveGate groups matching the source's purposes (routing, extensions, synthetic)	Verify group assignment in target tenant UI
4	Assign network zones if used — a separate setting from the group	Match source network zone structure
5	Verify connectivity from monitored hosts to new AGs	Test TCP connectivity on port **9999** (OneAgent → ActiveGate)

ActiveGate Network Zone Assignment

ActiveGate **groups** (which ActiveGates do which job) and **network zones** (which ActiveGates serve which agents) are separate settings. If the source tenant uses network zones to segment traffic, replicate the same zone structure in the target tenant — the zone names must match because migrated OneAgents keep their `--set-network-zone` value:

Source Zone	Purpose	Target Zone	ActiveGate Count
`us-east-prod`	Production workloads	`us-east-prod`	2 (HA pair)
`us-east-nonprod`	Dev/staging	`us-east-nonprod`	1
`eu-west-prod`	EU production	`eu-west-prod`	2 (HA pair)
`extensions`	Extension execution	`extensions`	1 (host-based only)

> **Extensions placement:** remote extensions run on an ActiveGate group; SQL monitoring extensions can alternatively run in Kubernetes through Dynatrace Operator (see the K8S series). Verify the supported runtime per extension before sizing the extensions zone.
>
> **Sources:** Extensions (DT docs) — *"Run SQL monitoring extensions on Kubernetes using Dynatrace Operator."*

Verify ActiveGate Connectivity

After deploying ActiveGates to the target tenant, verify they are reporting:

```
// Target tenant: verify ActiveGates are connected and reporting
smartscapeNodes "ACTIVEGATE"
| fields name, zone = dt.network_zone.id, group = dt.active_gate.group.name,
version = dt.active_gate.version
| sort name asc

// ActiveGates are their own Smartscape node type. Do not look for them among
hosts:
// containerized ActiveGates are not hosts at all, and host-based ones are
rarely named
// after their role, so a host-name filter returns zero rows on a tenant with
working AGs.
```

5. Kubernetes Operator Preparation

For environments running Kubernetes, the Dynatrace Operator must be configured to report to the target tenant. This involves updating the DynaKube custom resource and Helm values.

Preparation Steps

Step	Action	Notes
1	Generate new API token and PaaS token in target tenant	Scopes: `activeGateTokenManagement.create`, `entities.read`, `DataExport`, `metrics.read`
2	Create Kubernetes secret for target tenant	`kubectl create secret generic dynakube-target --from-literal=apiToken=<token> --from-literal=dataIngestToken=<token>`
3	Save the running source CR for rollback, then prepare the target CR	`kubectl get dynakube dynakube -n dynatrace -o yaml > dynakube-source.yaml`. Do **not** apply the target CR yet — wait for Step 5
4	Validate Helm chart version compatibility	Target tenant cluster version must support the operator version

DynaKube CR Template (Target Tenant)

```
apiVersion: dynatrace.com/v1beta6
kind: DynaKube
metadata:
  name: dynakube
  namespace: dynatrace
spec:
  apiUrl: https://&lt;target-env-id&gt;.live.dynatrace.com/api
  tokens: dynakube-target
  oneAgent:
    cloudNativeFullStack:
      tolerations:
        - effect: NoSchedule
          operator: Exists
  activeGate:
    capabilities:
      - routing
      - kubernetes-monitoring
      - dynatrace-api
    resources:
      requests:
        cpu: 500m
        memory: 512Mi
      limits:
        cpu: "1"
        memory: 1Gi
```

Helm Chart Preparation

```
# Inspect the Operator chart in the OCI registry
helm show chart oci://public.ecr.aws/dynatrace/dynatrace-operator --version
&lt;operator-version&gt;

# Dry-run to validate (do NOT install yet)
helm upgrade dynatrace-operator oci://public.ecr.aws/dynatrace/dynatrace-
operator \
  --version &lt;operator-version&gt; \
  --namespace dynatrace \
  --create-namespace \
  --install \
  --atomic \
  --dry-run
```

> **Do not apply the DynaKube CR yet.** Applying it now would cause agents to start reporting to the target tenant before configuration is imported. Wait for Step 5. The Helm chart installs the Operator only — the tenant URL and tokens live in the DynaKube CR and its secret, not in Helm values.

6. Configuration Freeze

A configuration freeze prevents drift between the exported configuration and the source tenant during the migration window. Changes made after the export but before import will be lost.

Freeze Scope

Category	Frozen?	Notes
Settings changes	Yes	No new alerting rules, detection rules, or dashboards
Management zone changes	Yes	No new zones or rule modifications
Auto-tag rule changes	Yes	No new tags or rule modifications
SLO changes	Yes	No new SLOs or metric expression changes
OneAgent deployments	No	New agent deployments continue (they auto-register)
Application deployments	No	Normal CI/CD continues
detected event generation	No	Monitoring remains fully active

Freeze Communication Template

Send to all Dynatrace users and stakeholders:

```
SUBJECT: Dynatrace Configuration Freeze — [DATE] to [DATE]

A configuration freeze is in effect for the Dynatrace source tenant
from [START DATE/TIME] to [END DATE/TIME].

During this period:
- Do NOT create or modify dashboards, SLOs, alerting rules, or settings
- Do NOT create or modify management zones or auto-tag rules
- Normal application deployments and monitoring continue as usual

Reason: SaaS-to-SaaS migration in progress. Configuration will be
imported to the target tenant and changes made after the export
snapshot will not be migrated.

Contact: [Migration Lead] for exceptions.
```

Monitoring for Freeze Violations

Use the audit log to detect any configuration changes made during the freeze window:

```
// Source tenant: detect configuration changes during freeze window
// Run this periodically during the freeze to catch violations
// Data object corrected 09/24/2026. The Dynatrace audit trail is NOT in
`logs`: the former
// `fetch logs | filter matchesPhrase(log.source, "audit")` matched nothing,
or matched an
// unrelated file-based audit log (a database .aud file on the validation
tenant). Environment
// audit records are structured events in `dt.system.events` with event.kind
== "AUDIT_EVENT".
// Covers Settings 2.0 objects (alerting, detection rules, management zones,
auto-tags, ...).
// Token operations are a separate provider (API_TOKEN), so they are already
excluded.
// Review user.id "UNKNOWN" rows before treating them as violations: on the
validation
// tenant they were metric-metadata and extension-definition writes.
fetch dt.system.events, from:-24h
| filter event.kind == "AUDIT_EVENT" and event.provider == "SETTINGS"
| filter in(event.type, {"CREATE", "UPDATE", "DELETE"})
| filter user.id != "system"
| summarize {changes = count(), last_change = takeMax(timestamp)}, by:
{user.id, details.dt.settings.schema_id}
| sort changes desc
| limit 20
```

7. Rollback Procedure

Every migration must have a documented rollback procedure. Rollback is possible at any point during Steps 4–6 because the source tenant remains fully operational.

Rollback Decision Criteria

Signal	Severity	Rollback?
Entity count < 80% of baseline	Critical	Yes — agents not connecting
No log data in target after 1 hour	Critical	Investigate first, rollback if unresolved in 2 hours
detected problems in target tenant unrelated to migration	Low	No — expected during transition
Dashboard shows no data	Medium	No — likely entity ID remapping issue (fixable)
IAM users cannot log in	Critical	Yes — SSO misconfiguration

Rollback Steps

Step	Action	Time Estimate
1	Re-run `oneagentctl` with the source server, tenant and tenant token (`--set-server` , `--set-tenant` , `--set-tenant-token` , `--restart-service`)	15 min per wave
2	Delete the target DynaKube and re-apply the source CR saved in Section 5 (`apiUrl` is immutable — it cannot be edited back)	5 min + pod restart
3	Verify agents reconnect to source tenant	15 min
4	Lift configuration freeze on source tenant	Immediate
5	Notify stakeholders	Immediate
6	Post-mortem and re-plan	Next business day

> **Key principle:** The source tenant is never decommissioned until the migration is fully validated (Step 9). Rollback is always possible by pointing agents back to the source tenant.

8. Step Completion Checklist

Before proceeding to Step 5 (Execute), verify all preparation tasks are complete:

Deliverable	Status	Owner	Notes
Target tenant provisioned	☐	Platform	Environment active, Grail enabled
SSO configured and tested	☐	Security / Platform	At least two group memberships validated
IAM deployed (Terraform or `monaco account`)	☐	Platform	Groups, policies, bindings applied
Monaco export completed	☐	Platform	Full export validated with dry-run
Entity ID audit completed	☐	Platform	Hardcoded IDs identified and remapping plan ready
ActiveGates deployed	☐	Platform / Infra	All zones covered, connectivity validated
K8s operator prepared	☐	Platform / K8s	DynaKube CR ready, secrets created, Helm validated
Configuration freeze active	☐	All	Freeze communicated, monitoring for violations active
Rollback procedure documented	☐	Platform	Reviewed and approved by migration lead
Pre-migration baseline recorded	☐	Platform	Entity counts, bucket inventory captured

> **Gate check:** Do not proceed to Step 5 until all items are checked. Missing preparation creates compounding problems during execution.

Next Step

Continue to **S2S-05: Step 5 — Execute: Configuration Import and Agent Cutover** to import configuration and redirect agents to the target tenant.

Completed	Next	Remaining
~~1. Discover~~ → ~~2. Strategize~~ → ~~3. Design~~ → ~~4. Prepare~~	**5. Execute**	6. Integrate → 7. Expand → 8. Enable → 9. Optimize

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.